Name: Vakkalanka Sai Bhaskara Sujit ID number: 2520090085

Week 4

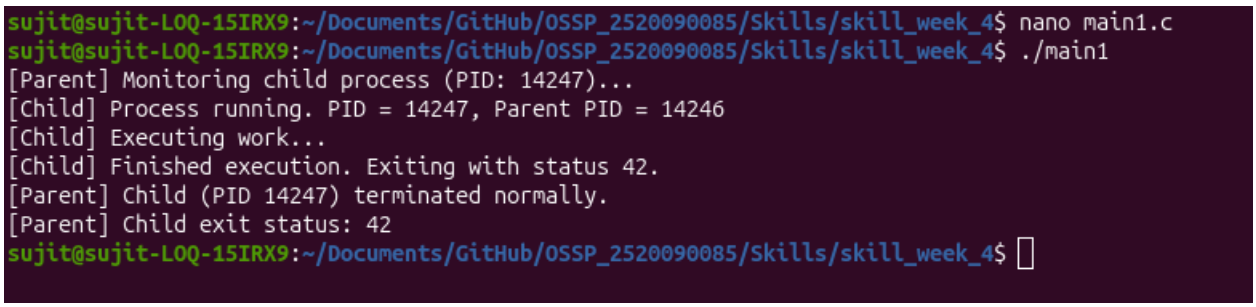
SESSION-7

Task 1: Process Synchronization and Child Process Monitoring Using waitpid()

To create a child process, synchronize the parent with the child using waitpid(), monitor the child process, check normal termination, retrieve the exit status, handle signal termination, and test process synchronization.

CODE:

OUTPUT:



Task 2: PATH Variable Retrieval and Command Resolution

To retrieve the PATH variable, parse its search directories, locate executable files, verify execute permissions, handle missing commands, and test command resolution.

CODE:

```
  GNU nano 8.7.1
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main(int argc, char *argv[]) {
    // 1. Input validation
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <command>\n", argv[0]);
        return 1;
    }

    char *cmd = argv[1];

    // 2. Retrieve the PATH environment variable
    char *path_env = getenv("PATH");
    if (path_env == NULL) {
        fprintf(stderr, "Error: PATH environment variable not found.\n");
        return 1;
    }

    printf("PATH = %s\n", path_env);

    // 3. Case A: User supplied an explicit relative or absolute path (contains
    if (strchr(cmd, '/') != NULL) {
        if (access(cmd, F_OK) == 0) {
            printf("File found: %s\n", cmd);
            if (access(cmd, X_OK) == 0) {
                printf("Execute permission: YES\n");
                printf("Command resolved successfully.\n");
                return 0;
            } else {
                printf("Execute permission: NO\n");
                printf("Command not found or not executable\n");
                return 1;
            }
        } else {
            printf("Command not found or not executable\n");
            return 1;
        }
    }

    // 4. Case B: Search through each directory listed in PATH
    char *path_copy = strdup(path_env);
    if (path_copy == NULL) {
        perror("strdup failed");
        return 1;
    }

    char *dir = strtok(path_copy, ":");
    char full_path[PATH_MAX];
    int found = 0;

    while (dir != NULL) {
        snprintf(full_path, sizeof(full_path), "%s/%s", dir, cmd);

        // Check if file exists and has executable permission
        if (access(full_path, F_OK) == 0) {
            if (access(full_path, X_OK) == 0) {
                printf("File found: %s\n", full_path);
                printf("Execute permission: YES\n");
                printf("Command resolved successfully.\n");
                found = 1;
                break;
            }
        }
        dir = strtok(NULL, ":");
    }

    // 5. Handle missing commands
    if (!found) {
        printf("Command '%s' not found in PATH.\n", cmd);
    }

    // 6. Clean up resources
    free(path_copy);
    return found ? 0 : 1;
}
```

OUTPUT:

```
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$ nano task2.c
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$ ./task2 pwd
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
File found: /usr/bin/pwd
Execute permission: YES
Command resolved successfully.
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$ ./task2 ls
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
File found: /usr/bin/ls
Execute permission: YES
Command resolved successfully.
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$
```

SESSION-8

Task 1: Variable Reference Detection and Value Expansion

To detect variable references, expand environment variable values, handle undefined variables, update tokens after expansion, support nested variables, and test expansion logic.

CODE:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_ARGS 64
#define BUFFER_SIZE 1024

// Command function declarations
int builtin_cd(char **args);
int builtin_pwd(char **args);
int builtin_echo(char **args);
int builtin_exit(char **args);

// Structure definition for dispatch table
typedef struct {
    const char *name;
    int (*func)(char **args);
} BuiltinCommand;

// Command dispatch lookup table
BuiltinCommand dispatch_table[] = {
    {"cd",   builtin_cd},
    {"pwd",  builtin_pwd},
    {"echo", builtin_echo},
    {"exit", builtin_exit},
    {NULL,   NULL}
};

int builtin_cd(char **args) {
    if (args[1] == NULL) {
        char *home = getenv("HOME");
        if (home != NULL) {
            chdir(home);
        }
    } else {
        if (chdir(args[1]) != 0) {
            perror("cd");
        }
    }
    return 1;
}

int builtin_pwd(char **args) {
    (void)args;
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd");
    }
    return 1;
}

int builtin_echo(char **args) {
    for (int i = 1; args[i] != NULL; i++) {
```

```c
    return 1;
}

int builtin_exit(char **args) {
    (void)args;
    printf("Exiting shell...\n");
    exit(0);
}

int main(void) {
    char line[BUFFER_SIZE];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            break;
        }

        // Remove newline character
        line[strcspn(line, "\n")] = '\0';

        // Tokenize command string
        int arg_count = 0;
        char *token = strtok(line, " \t");
        while (token != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = token;
            token = strtok(NULL, " \t");
        }
        args[arg_count] = NULL;

        // Skip empty command line
        if (args[0] == NULL) {
            continue;
        }

        // Search dispatch table
        int executed = 0;
        for (int i = 0; dispatch_table[i].name != NULL; i++) {
            if (strcmp(args[0], dispatch_table[i].name) == 0) {
                dispatch_table[i].func(args);
                executed = 1;
                break;
            }
        }

        // Handle invalid commands
        if (!executed) {
            printf("Command not found: %s\n", args[0]);
        }
    }

    return 0;
}
```

OUTPUT:

```
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$ ./task3
Enter a string: Sujit
Original : Sujit
Expanded : Sujit
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$ ./task3
Enter a string: hello $abc
Warning: Undefined variable: abc
Original : hello $abc
Expanded : hello
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$ ./task3
Enter a string: Enter a string: Current user is $USER
Original : Enter a string: Current user is $USER
Expanded : Enter a string: Current user is sujit
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$
```

Task 2: Built-in Identification, Dispatch Table and In-Process Execution

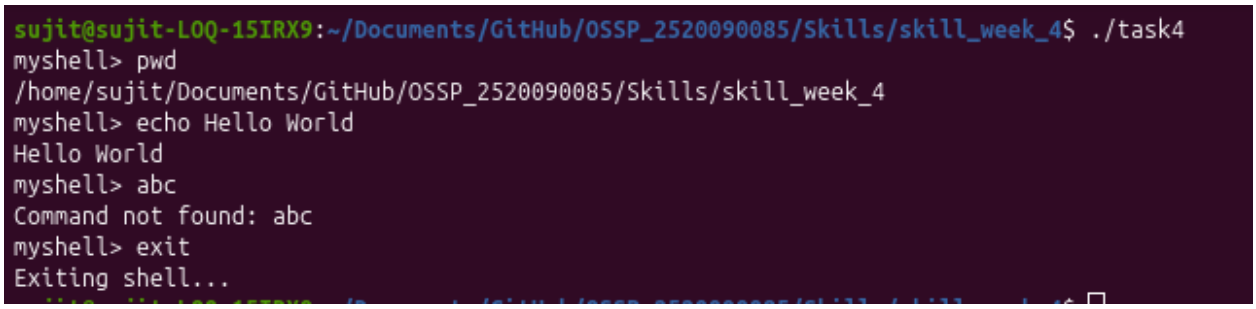
To identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain shell state, and test dispatch logic.

CODE:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_ARGS 64
#define BUFFER_SIZE 1024

// Command function declarations
int builtin_cd(char **args);
int builtin_pwd(char **args);
int builtin_echo(char **args);
int builtin_exit(char **args);

// Structure definition for dispatch table
typedef struct {
    const char *name;
    int (*func)(char **args);
} BuiltinCommand;

// Command dispatch lookup table
BuiltinCommand dispatch_table[] = {
    {"cd",   builtin_cd},
    {"pwd",  builtin_pwd},
    {"echo", builtin_echo},
    {"exit", builtin_exit},
    {NULL,   NULL}
};

int builtin_cd(char **args) {
    if (args[1] == NULL) {
        char *home = getenv("HOME");
        if (home != NULL) {
            chdir(home);
        }
    } else {
        if (chdir(args[1]) != 0) {
            perror("cd");
        }
    }
    return 1;
}

int builtin_pwd(char **args) {
    (void)args;
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd");
    }
    return 1;
}

int builtin_echo(char **args) {
    for (int i = 1; args[i] != NULL; i++) {
```

```
  GNU nano 8.7.1
}

int builtin_exit(char **args) {
    (void)args;
    printf("Exiting shell...\n");
    exit(0);
}

int main(void) {
    char line[BUFFER_SIZE];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            break;
        }

        // Remove newline character
        line[strcspn(line, "\n")] = '\0';

        // Tokenize command string
        int arg_count = 0;
        char *token = strtok(line, " \t");
        while (token != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = token;
            token = strtok(NULL, " \t");
        }
        args[arg_count] = NULL;

        // Skip empty command line
        if (args[0] == NULL) {
            continue;
        }

        // Search dispatch table
        int executed = 0;
        for (int i = 0; dispatch_table[i].name != NULL; i++) {
            if (strcmp(args[0], dispatch_table[i].name) == 0) {
                dispatch_table[i].func(args);
                executed = 1;
                break;
            }
        }

        // Handle invalid commands
        if (!executed) {
            printf("Command not found: %s\n", args[0]);
        }
    }

    return 0;
}
S
```

OUTPUT:


sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4$ ./task4
myshell> pwd
/home/sujit/Documents/GitHub/OSSP_2520090085/Skills/skill_week_4
myshell> echo Hello World
Hello World
myshell> abc
Command not found: abc
myshell> exit
Exiting shell...